Desenvolvimento de um Gerador de Lances para Xadrez em Rust com Bitboards e Magic Bitboards

Guilherme Borges Pagano
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
guilhermebpagano@ufu.br

João Marcos de Oliveira Calixto
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
jm.calixto@ufu.br

Augusto W. F. Veloso da Silveira
Faculdade de Engenharia Elétrica
Universidade Federal de Uberlândia
Uberlândia, Brasil
augustofleury@ufu.br

***Resumo* — A aplicação de inteligência artificial ao xadrez exige estruturas de dados e algoritmos de alta eficiência, uma vez que a geração de lances e a exploração de árvores de busca impactam diretamente o desempenho do sistema. Este artigo apresenta o desenvolvimento da ChessLib, uma biblioteca para xadrez implementada em Rust, voltada à representação eficiente do tabuleiro e à geração de lances com base em bitboards. A solução emprega operações bitwise e magic bitboards para otimizar a manipulação do estado do jogo e o cálculo de movimentos, com foco em desempenho, segurança de memória e modularidade. Além disso, o trabalho propõe uma base sólida para futuras aplicações de inteligência artificial em xadrez, oferecendo uma arquitetura preparada para expansão e integração com algoritmos de busca.**

***Palavras chave*: Rust, Xadrez, Inteligência Artificial, Bitboards, Geração de lances;**

I. Introdução

A interseção entre xadrez e inteligência artificial tem sido um campo de estudo relevante por décadas, servindo como referência para avanços em busca computacional, heurísticas de avaliação e aprendizado de máquina.

> Desde as vitórias do Deep Blue da IBM sobre Garry Kasparov até a ascensão de engines baseados em redes neurais como o AlphaZero e o Leela Chess Zero, a capacidade de uma máquina de superar os melhores jogadores humanos tornou-se uma realidade.
>
> — D. Silver *et al.* [1]

No núcleo de qualquer engine de xadrez está um componente fundamental: a camada responsável por representar o tabuleiro, validar regras e gerar lances de maneira correta e eficiente. A eficiência dessa camada é especialmente importante porque afeta diretamente o custo computacional da busca em profundidade.

Este trabalho apresenta o desenvolvimento da ChessLib, uma biblioteca de xadrez implementada em Rust, com foco em desempenho, segurança de memória e modularidade. A proposta utiliza bitboards como representação do tabuleiro e magic bitboards para otimizar a geração de lances de peças deslizantes. Assim, busca-se oferecer uma base confiável para aplicações futuras de inteligência artificial em xadrez, reduzindo gargalos na camada de geração de movimentos.

A principal contribuição deste artigo é a descrição da arquitetura da biblioteca, da estratégia de implementação e da metodologia de avaliação adotada para verificar sua corretude e eficiência.

II. Trabalhos Relacionados

O desenvolvimento de softwares de xadrez pode ser dividido em duas frentes principais: engines completas, projetadas para jogar autonomamente, e bibliotecas de lógica do jogo, responsáveis por abstrair regras, estado do tabuleiro e geração de lances.

A. Engines de Xadrez de Alta Performance

O engine de xadrez de código aberto mais forte da atualidade é o Stockfish, desenvolvido em C++. Sua força deriva de uma busca alfa-beta altamente otimizada [2]. O Leela Chess Zero (Lc0), por sua vez, representa a abordagem baseada em aprendizado de máquina, utilizando uma rede neural para guiar uma busca Monte Carlo Tree Search (MCTS) [3].

B. Bibliotecas de Lógica de Xadrez

Além de engines completas, existem bibliotecas voltadas à representação e manipulação do jogo. Essas soluções são úteis tanto para prototipagem quanto para integração em sistemas maiores. Em Python, por exemplo, a biblioteca python-chess é amplamente utilizada pela simplicidade e pela cobertura funcional. Contudo, em aplicações que exigem grande volume de geração de lances, linguagens interpretadas tendem a apresentar limitações de desempenho.

Nesse contexto, Rust surge como uma alternativa interessante por combinar alto desempenho com segurança de memória e ausência de coletor de lixo. Essas características tornam a linguagem adequada para componentes centrais de engines de xadrez, especialmente aqueles que exigem controle fino sobre alocação, acesso à memória e operações em nível de bits.

A ChessLib insere-se nesse cenário como uma implementação voltada especificamente à eficiência da geração de lances, buscando unir desempenho e robustez em uma base modular para futuros motores de xadrez.

III. Arquitetura

A arquitetura da ChessLib, projetada para modularidade e performance, inspira-se em implementações de alto desempenho,

como a da biblioteca “chess” [4]. Sua estrutura é fundamentada na representação do tabuleiro por meio de Bitboards, na utilização de operações Bitwise e na geração de lances, com destaque para o uso da técnica de Magic Bitboards na geração de lances avançados.

A. Bitboards

A eficiência de um engine de xadrez depende diretamente da forma como o estado do jogo é representado. Em vez de estruturas bidimensionais tradicionais, a abordagem por bitboards representa o tabuleiro como um conjunto de inteiros de 64 bits, nos quais cada bit corresponde a uma casa do tabuleiro.

Essa representação permite mapear diretamente as 64 casas do tabuleiro para posições de bits em uma palavra de máquina, aproveitando operações nativas da CPU para manipulação simultânea de múltiplas casas. Assim, o estado do jogo pode ser armazenado de forma compacta e processado com grande eficiência.

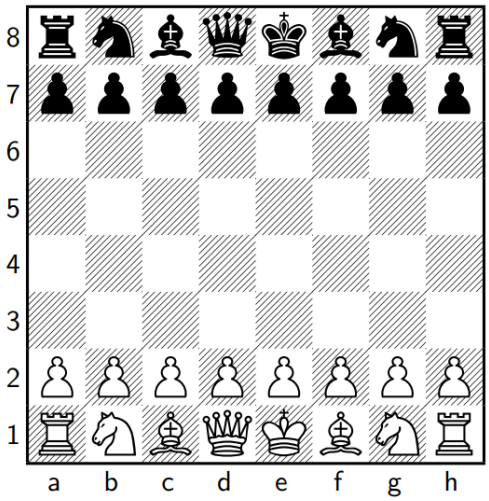

Fig. 1: Tabuleiro de Xadrez

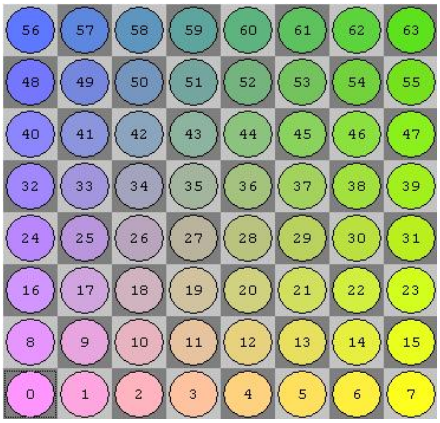

Fig. 2: Little-Endian File Mapping

Na convenção mais comum, chamada Little-Endian File Mapping (Fig. 2), a casa “a1” é associada ao bit menos significativo, enquanto h8 corresponde ao bit mais significativo. Com isso, o índice de cada casa varia de 0 a 63, facilitando deslocamentos e operações aritméticas sobre o tabuleiro.

Um estado completo do jogo pode ser representado por múltiplos bitboards, normalmente um para cada tipo de peça em cada cor. A partir desses bitboards primários, também podem ser derivados conjuntos auxiliares, como todas as casas ocupadas por peças brancas, por peças pretas ou pelo tabuleiro inteiro. Esses bitboards agregados simplificam verificações de ocupação e ataques.

B. Operações Bitwise

A manipulação de bitboards depende de operações bitwise, como AND, OR, XOR, NOT e deslocamentos à esquerda e à direita. Essas operações são extremamente eficientes, pois atuam sobre 64 bits simultaneamente.

Por exemplo, o avanço simples de peões brancos pode ser calculado por meio de um deslocamento de 8 bits à esquerda, seguido da interseção com as casas vazias:

```
avancos_possiveis = (peoes_brancos << 8) & casas_vazias
```

Nesse caso, o deslocamento representa o avanço de uma fileira no tabuleiro, enquanto a operação AND filtra apenas os destinos realmente disponíveis.

C. Geração de Lances para Peças de Passo:

As peças de passo são aquelas cujo deslocamento segue padrões fixos e relativamente simples de calcular. Nesse grupo estão o peão, o cavalo e o rei.

Essas peças apresentam menor complexidade de geração em comparação com as peças deslizantes, pois seus movimentos podem ser pré-calculados ou obtidos por meio de operações bitwise diretas.

a) *Geração de Lances para Cavalo e Rei:*

Os movimentos do cavalo e do rei dependem apenas da casa de origem e da ocupação da casa de destino. Por esse motivo, é possível pré-calcular os ataques válidos para cada uma das 64 casas e armazená-los em tabelas de consulta.

Em tempo de execução, a geração de lances consiste em acessar a tabela correspondente e aplicar uma máscara para remover casas ocupadas por peças da mesma cor.

Um desafio na pré-computação desses movimentos é o problema do “wrap-around”, onde um deslocamento de bits pode fazer uma peça “saltar” de uma borda do tabuleiro para a outra (*e.g.*, um cavalo em h1, ao se mover duas casas para cima e uma para a esquerda, poderia erroneamente pousar em a3). Isso é evitado usando máscaras de arquivo. Por exemplo, para um movimento que se desloca para a direita, o bitboard da peça é primeiro combinado com uma máscara que zera os bits do arquivo ‘h’ antes do deslocamento, garantindo que nenhuma peça no arquivo ‘h’ possa “envolver” o tabuleiro.

b) *Geração de lances para o Peão:*

Os peões possuem regras de movimentação mais particulares, envolvendo avanço simples, avanço duplo, capturas diagonais, promoção e en passant.

O avanço simples é obtido por deslocamento de 8 bits e filtragem pelas casas vazias. O avanço duplo é permitido apenas a partir da posição inicial do peão e exige que ambas as casas intermediárias estejam desocupadas.

As capturas diagonais são calculadas por deslocamentos de 7 e 9 bits, também combinados com máscaras para evitar wrap-around. Já a promoção ocorre quando o peão alcança a última fileira do adversário, gerando lances distintos para as peças de promoção possíveis.

Por fim, a captura *en passant* requer que o estado do jogo armazene informações sobre o último lance do oponente. Especificamente, se o último lance foi um avanço duplo de peão, a casa de destino é registrada como uma casa de *en passant* potencial. A geração de lances de captura en passant envolve verificar se um peão próprio ataca essa casa especial. Se um lance de captura en passant é feito, a lógica de atualização do tabuleiro deve remover manualmente o peão capturado, que está em uma casa adjacente à casa de destino do peão que captura.

D. *Geração de Lances para Peças Deslizantes:*

A geração de lances para peças deslizantes (torres, bispos e damas) representa o desafio mais significativo na implementação de um gerador de lances com bitboard. O movimento dessas peças não é fixo; ele se estende ao longo de raios (fileiras, colunas e diagonais) até encontrar a borda do tabuleiro ou outra peça. A determinação desses movimentos depende, portanto, da configuração de "bloqueadores" em seus caminhos.

Uma abordagem ingênua de iterar ao longo de cada raio para cada peça seria extremamente lenta. Para resolver este problema, a comunidade de programação de xadrez desenvolveu várias técnicas sofisticadas, entre elas, os Magic Bitboards, que receberão uma atenção especial a seguir.

E. *Magic Bitboards*

O problema central consiste em mapear cada configuração possível de bloqueadores para o conjunto correspondente de ataques válidos. Como o número de configurações possíveis é grande, mas o número de ataques distintos é bem menor, essa relação pode ser compactada por meio de tabelas pré-computadas.

A técnica de magic bitboards utiliza uma função de hash do tipo multiplicação e deslocamento:

```
index = (blocker_board * magic_number) » shift_amount
```

Onde:

- Blocker Board é um bitboard contendo apenas os bloqueadores relevantes da peça.
- Magic Number é uma constante de 64 bits cuidadosamente escolhida, cujos bits estão dispostos de tal forma que, quando multiplicada por qualquer permutação válida do Blocker Board, os bits de ordem superior do resultado de 64 bits são determinados de forma única pela permutação.
- Shift Amount vai isolar esses bits de ordem superior, descartando os bits inferiores e menos úteis do resultado da multiplicação. O resultado é um índice compacto para uma tabela de consulta.

Essa abordagem permite armazenar apenas os ataques distintos em tabelas de consulta, reduzindo custo de tempo em tempo de execução.

a) *Arquitetura dos Magic Bitboards:*

A implementação depende de três elementos principais:

Máscara de bloqueadores:

A máscara de bloqueadores é a primeira componente essencial. Para uma dada peça numa dada casa, é um bitboard com bits definidos para cada casa nos raios de ataque da peça, excluindo a própria casa da peça e as casas na extremidade do tabuleiro. O número de bits definidos nesta máscara determina o tamanho da tabela de consulta para essa casa, que será de 2^{bits} entradas.

Uma otimização importante é a "exclusão de borda", onde as casas na extremidade de cada raio são excluídas da máscara, pois um ataque à casa final só é possível se a casa penúltima estiver vazia. Portanto, o estado da casa final é redundante para determinar o conjunto de ataques.

Número Mágico::

O número mágico é uma constante de 64 bits, única para cada casa e tipo de peça (torre/bispo), que foi descoberta através de uma busca por força bruta para satisfazer a propriedade de hashing perfeito para a máscara de bloqueadores dessa casa.

Estes números não são derivados de uma fórmula matemática, mas são encontrados através de um processo de tentativa e erro. A comunidade de programação de xadrez mantém listas dos "melhores mágicos até agora", que são números que não só funcionam, mas também permitem tabelas de ataque mais compactas.

Cada uma das 128 combinações (64 para torres, 64 para bispos) tem o seu próprio número mágico único.

Tabela de Ataques::

É um grande array (ou múltiplos arrays) que armazena os conjuntos de ataques pré-calculados. O índice para esta tabela é o valor gerado pela fórmula mágica. O valor nesse índice é um bitboard de 64 bits que representa todos os movimentos possíveis para a configuração de bloqueadores dada.

b) *Busca pelos Magic Numbers:*

A etapa de inicialização consiste em encontrar números mágicos válidos para cada casa e cada tipo de peça deslizante. Esse processo é realizado uma única vez, durante a inicialização ou previamente ao uso da biblioteca.

O processo de geração pode ser conceptualizado como uma série de ciclos aninhados: para cada uma das 64 casas, e para cada tipo de peça deslizante (torre e bispo), a engine deve encontrar um número mágico funcional. O algoritmo para encontrar um único número mágico é o disposto na Fig. 3.

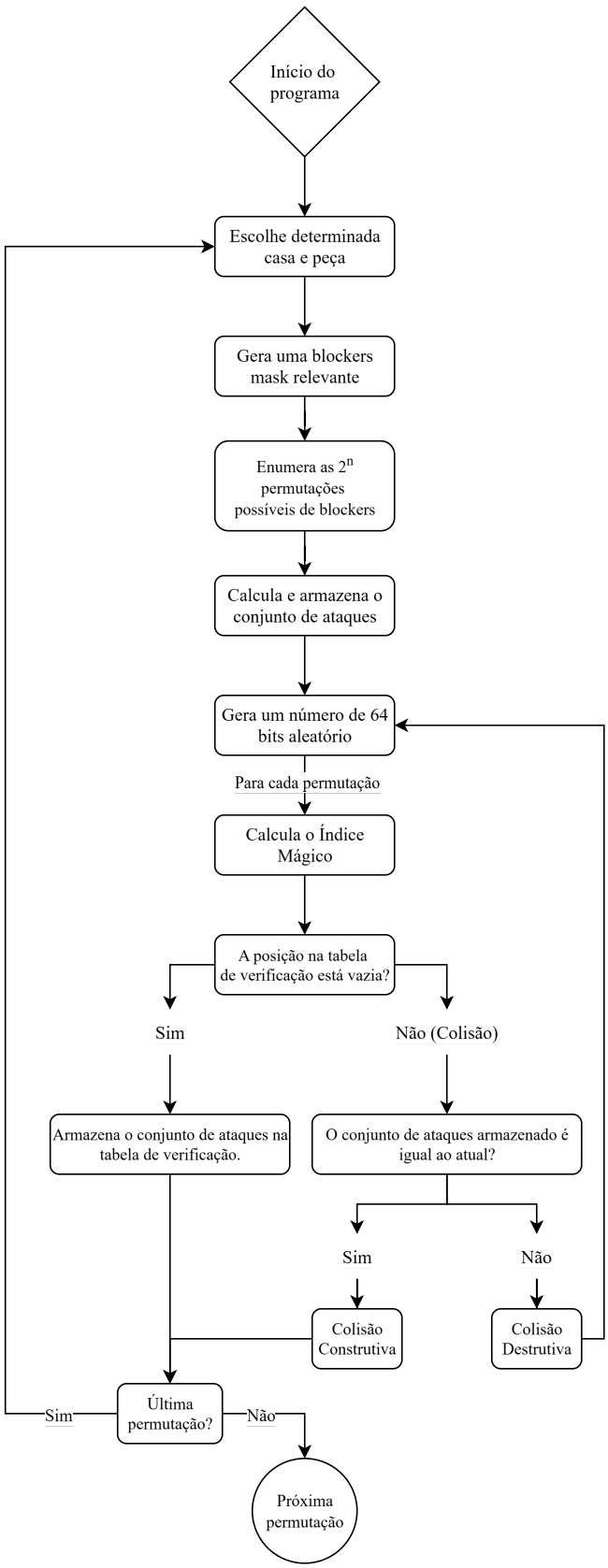

Fig. 3: Geração e Validação de um Número Mágico

Uma vez inicializado, o processo de geração de movimentos em tempo de execução é extremamente eficiente, consistindo em uma sequência linear de operações apresentadas na Fig. 4.

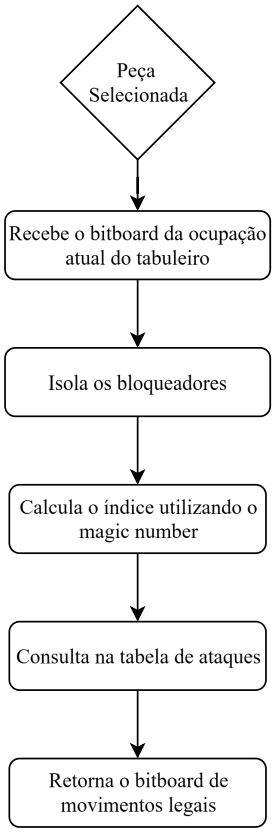

Fig. 4: Geração e Validação de um Número Mágico

IV. Avaliação Experimental

A. *Teste de Desempenho (Perft)*

O teste Perft é uma técnica amplamente utilizada para validação de engines de xadrez. Ele percorre a árvore de lances até uma profundidade especificada e contabiliza o número total de nós gerados.

Além de medir desempenho, o Perft também é útil para verificar a corretude da geração de lances, já que contagens conhecidas podem ser comparadas com os valores produzidos pela implementação.

B. *Especificações do Sistema e do Software*

Os resultados de desempenho são dependentes do hardware em que são executados. A Tabela I documenta o ambiente utilizado nos testes:

Tabela I: Especificações do sistema

Componente	Especificação
CPU	Intel Core i7-12800H @ 4.70GHz
Núcleos/Threads	14 Cores / 20 Threads
RAM	32 GB
OS	Arch Linux x86_64 6.12.42-1-lts

C. Bibliotecas sob teste

As versões específicas das bibliotecas utilizadas neste estudo estão dispostas na Tabela II:

Tabela II: Especificações das bibliotecas

Lib	Versão
ChessLib	-
Python-Chess	-
Chess	-

D. Testes utilizados e Procedimento

Foi utilizado um conjunto curado de posições de teste, representadas na Notação Forsyth-Edwards (FEN), para garantir uma avaliação abrangente que testa diferentes aspetos da lógica de geração de lances:

a) *Posição Inicial:*

Linha de base padrão para testes Perft.

`rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1`

b) *"Kiwipete":*

Uma posição complexa de meio-jogo com muitas possibilidades táticas, incluindo roques, capturas e lances de peão. Testa o desempenho num cenário mais realista e computacionalmente denso.

`r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBBPPP/R3K2R w KQkq - 0 1`

c) *Teste de promoção:*

Uma posição projetada especificamente para testar a lógica de promoção de peões, que pode ser uma fonte de bugs e ineficiências.

`n1n5/PPPk4/8/8/8/8/4Kppp/5N1N w - - 0 1`

Cada biblioteca executou um teste Perft em cada uma das quatro posições, começando na profundidade 1 e continuando até uma profundidade computacionalmente significativa (Profundidade 6 para a Posição Inicial, Profundidade 5 para as outras).

V. Resultados

O primeiro passo é estabelecer que os sistemas sob teste estão a produzir resultados funcionalmente corretos. A Tabela III compara a contagem de nós gerada por cada biblioteca para a posição inicial com os valores canónicos estabelecidos, amplamente aceitos pela comunidade de programação de xadrez.

Tabela III: Validação da Correção do Perft (Posição Inicial)

Profundidade	Contagem Canônica de Nós	ChessLib	Python-Chess	Chess	Resultado
1	20	-	-	-	-
2	400	-	-	-	-
3	8902	-	-	-	-
4	197281	-	-	-	-
5	4865609	-	-	-	-

A análise confirma que todas as três bibliotecas geram o número correto de nós folha até uma profundidade de 5. Este resultado estabelece um alto grau de confiança na correção das suas respetivas implementações de geração de lances.

Com a correção validada, a análise foca-se na eficiência computacional. A Tabela IV apresenta os resultados de desempenho para a posição inicial, mostrando o tempo de execução e a métrica de desempenho Nós Por Segundo (NPS).

Tabela IV: Benchmark de Desempenho (Posição Inicial)

Lib	Lang	Profundidade	Nós	Tempo [s]	NPS
Python-Chess	Python	5	-	-	-
Python-Chess	Python	6	-	-	-
ChessLib	Rust	5	-	-	-
ChessLib	Rust	6	-	-	-
Chess	Rust	5	-	-	-
Chess	Rust	6	-	-	-

Para garantir que as características de desempenho observadas não são um artefacto da posição inicial, os testes foram repetidos em posições mais complexas e variadas. A Tabela V resume o desempenho (em NPS) para cada biblioteca na profundidade 5 para as posições de teste "Kiwipete", "Teste de Promoção" e "Muitas Capturas".

Tabela V: Benchmark de Desempenho, Profundidade 5 (Posições Complexas)

Posição de teste	NPS (Python-Chess)	NPS (ChessLib)	NPS (Chess)
Kiwipete	-	-	-
Teste de promoção	-	-	-
Muitas Capturas	-	-	-

VI. Discussão

VII. Conclusão

Referências

[1] D. Silver *et al.*, "Mastering the game of go without human knowledge", 2017, *Nature Publishing Group*.

[2] The Stockfish Team, "Stockfish Chess Engine". Acesso em: 23 de junho de 2025. [Online]. Disponível em: https://stockfishchess.org/

[3] The Leela Chess Zero Team, “Leela Chess Zero”. Acesso em: 23 de junho de 2025. [Online]. Disponível em: https://lczero.org/

[4] J. Bray, “chess: A pure Rust chess library”. Acesso em: 23 de junho de 2025. [Online]. Disponível em: https://github.com/jordanbray/chess